

Data Engineer [Take Home Test]

Option C: Cross-Portfolio Revenue Attribution & Forecasting Pipeline

Time Expectation: 8 – 10 hours Complexity: 8.5/10

Note: Please make sure to only submit the take home test via GitHub and provide the url.

Forethought

We believe the true essence of data engineering extends far beyond writing ETL scripts and SQL queries. In crafting this take-home exercise, we've designed it to reflect the authentic challenges a data engineer encounters when building end-to-end data solutions across a portfolio of businesses — messy sources, ambiguous schemas, stakeholders who need answers yesterday.

As practicing engineers ourselves, we understand that no one is an expert in every tool or cloud platform. The reality of our profession is that we constantly navigate unfamiliar data sources, learn new tools, and solve complex data problems through research, experimentation, and pragmatic decision-making. This exercise is designed to honor that reality.

We embrace the modern data engineering landscape where AI-powered tools like GitHub Copilot, ChatGPT, and Claude have become valuable allies. We **strongly encourage** you to leverage these tools, just as you would in your daily work. Your ability to effectively research, implement data solutions, and deliver insights matters more to us than any pre-existing knowledge.

When evaluating your submission, we'll focus on your *engineering approach* — how you structure your data architecture, the clarity of your code, the thoughtfulness of your documentation, and the reasoning behind your technical decisions. We care about seeing your authentic problem-solving process rather than a perfect implementation.

Take this opportunity to show us how you think, learn, and create as a data engineer. Document your journey, explain your choices, and don't hesitate to make pragmatic decisions about where to focus your efforts. We're looking forward to understanding your unique perspective and approach to data engineering through this exercise. We're not looking for a perfect solution!

Overview

Build an end-to-end analytics pipeline that ingests content performance data, portfolio company revenue data, and user journey/attribution data, then produces a revenue attribution model linking content channels to portfolio revenue. Include a basic forecasting layer using SQL or Python.

Business Context

Alpha Tango's media arm (5M+ social followers, 100M+ monthly views) is the acquisition engine for its portfolio companies. Content drives traffic, traffic drives signups, signups drive revenue across portfolio businesses. But nobody can answer the question: **"Which content is actually driving revenue, and how much?"**

Marketing says social is the growth lever. The portfolio companies say organic and referral. Leadership wants data, not opinions. You're building the pipeline that connects the dots from content → traffic → signup → revenue, and gives leadership an attribution model they can actually use.

Starter Data

You'll receive a starter repository with pre-generated sample data files across 4 sources:

Source	Format	Records	Description
Content Performance	CSV	~1,370	Content metrics by channel — views, clicks, UTMs
User Signups	JSON	~6,170	Signup events with attribution (first/last touch, UTMs)
Portfolio Revenue	CSV	~27,070	Monthly revenue per user per company
Campaign Metadata	CSV	30	Campaign details — channel, budget,

Source	Format	Records	Description
			dates, target company

Channels

YouTube, Twitter/X, Instagram, Newsletter, Podcast, Blog (+ organic & referral in signups)

Portfolio Companies

ID	Name	Industry	Avg Revenue/User
CTC-001	GreenLeaf Landscaping	Home Services	~\$120
CTC-002	BrightPath Education	Education	~\$85
CTC-003	QuickFix Auto Repair	Automotive	~\$210
CTC-004	Summit Dental Group	Healthcare	~\$165

Data Files

data/

|— content_performance.csv # content_id, channel, views, clicks, UTMs

|— user_signups.json # signup + attribution data (first/last touch)

|— portfolio_revenue.csv # revenue by user by month

|— campaign_metadata.csv # budget, dates, channel, target company

Known Data Quality Characteristics

- ~8% of content records have missing `utm_source`
- ~12% of users have missing UTM source
- ~15% of users have missing campaign UTMs
- Some users have conflicting `first_touch_channel` and `last_touch_channel`
- Some users have `referral_source` that doesn't match their UTM data
- Revenue spans multiple months per user (recurring vs one-time)

Data is deterministic (seeded random). You can regenerate anytime with `python generate_data.py`.

Functional Requirements

MUST — Data Ingestion & Loading

- Python scripts to ingest all sources into raw/staging layer
- Data validation: UTM consistency checks, referential integrity (user_ids exist across tables), date range validation
- Handle messy attribution data — some users have no utm params, some have conflicting first/last touch
- Load into BigQuery (or DuckDB/PostgreSQL for local dev)

MUST — Transformation & Modeling (dbt)

- **Staging:** Clean models per source
- **Intermediate:** User journey stitching — join signup attribution to revenue records. Resolve conflicting attribution (document your logic for handling missing/conflicting UTMs)
- **Mart models:**
 - `fct_attributed_revenue` — revenue records enriched with attribution channel, campaign, content_id
 - `fct_content_performance` — content metrics aggregated by channel and time period
 - `fct_campaign_roi` — campaign spend vs attributed revenue
 - `dim_channel`, `dim_company`, `dim_user_cohort` (signup month cohort), `dim_time`
- Implement **two attribution models** (document the trade-offs):
 - Last-touch attribution
 - One other model of your choice (first-touch, linear, time-decay, or position-based)
- dbt tests on attribution logic, referential integrity, and metric consistency

MUST — Analytics

SQL queries answering:

1. **Channel revenue ranking:** Which content channels drive the most attributed revenue? (under both attribution models — show how the answer changes)
2. **Campaign ROI:** For each campaign, what was the spend vs attributed revenue? Rank by efficiency.
3. **Customer Acquisition Cost (CAC):** CAC by channel per company

4. **Cohort retention:** How does revenue retention look for users acquired via different channels?
5. **Revenue trend:** Month-over-month growth by channel, with a simple trailing-average forecast for the next 3 months

MUST — Forecasting

- A simple revenue forecast for the next 3 months per company — can be SQL-based (moving averages, linear regression approximation) or Python-based (statsmodels, simple exponential smoothing)
- Document your approach, its limitations, and when it would break down

OPTIONAL — Stretch Goals

- A Streamlit dashboard showing attribution comparison (last-touch vs your chosen model)
- Anomaly detection: flag months where a channel's performance deviates significantly from trend
- Data lineage documentation end-to-end
- Pipeline orchestration with Airflow/Prefect DAG

Technical Requirements

- **Pipeline:** Python 3.10+
- **Transformations:** dbt
- **Warehouse:** BigQuery preferred. DuckDB or PostgreSQL acceptable.
- **Forecasting:** SQL window functions or Python (pandas/statsmodels/scipy)
- **SQL:** Analytical queries in standard SQL

Expected Deliverables

1. Git repository with all code
2. Sample data files (included in starter repo)
3. Python ingestion scripts
4. dbt project with full model layering and tests
5. Attribution model implementation with comparison output
6. Forecasting implementation with documented methodology
7. SQL analytics queries with sample output
8. **README.md** with:
 - Setup instructions
 - Architecture diagram

- Data model documentation
 - Attribution model trade-off analysis
 - Forecasting methodology & limitations
 - Assumptions & trade-offs
 - Future improvements
-

Evaluation Criteria

Area	Weight	What We're Looking At
Attribution Modeling	25%	Do your attribution models work? Can you explain the differences in output and when to use each?
Data Modeling & dbt	25%	Model layering, naming, tests, handling of messy joins across sources.
Pipeline Quality	20%	Ingestion, validation, handling of dirty/missing attribution data.
Analytical Depth	20%	Do the queries tell a story? Is the forecast reasonable? Can you explain what the data says?
Documentation	10%	Trade-off analysis, methodology explanation, clear setup instructions.

Suggested Time Allocation

Phase	Time
Data exploration & project setup	1.5 – 2 hrs
Python ingestion pipeline	1.5 – 2 hrs

Phase	Time
dbt models & attribution logic	2.5 – 3 hrs
Analytics SQL & forecasting	1.5 – 2 hrs
Documentation & cleanup	1 hr

Submission

- Share a GitHub/GitLab repository link to [\[hiring-email@company.com\]](mailto:hiring-email@company.com)
- Ensure the repository is public or provide access to the above email if private
- Include a working local setup — we will run your code

Notes

- **Focus on getting a working pipeline end-to-end** rather than a perfect architecture
- Document any shortcuts taken due to time constraints
- Feel free to use any libraries, AI tools, agents, or anything on the internet that speeds up development
- Prioritize functionality, testability & readability over perfect code
- If you have any questions, feel free to reach out

What We're Looking For

- With the internet at your fingertips, *how fast can you think → learn → execute & deliver and repeat the cycle*
- Can you take messy, real-world data and turn it into something a business leader can use?
- Do you understand the difference between "data loaded" and "data modeled"?
- Can you model attribution in a way that's useful, not just technically correct?
- Can you write SQL that tells a story, not just returns rows?
- Do you document your thinking — the why, not just the what?
- Practical problem-solving within time constraints
- Clear documentation of your approach, decisions, and trade-offs

A Note on AI Tools

We genuinely mean it — use Copilot, ChatGPT, Claude, Cursor, or whatever makes you productive. We care about what you deliver and how you think, not whether you typed every

character. If an AI tool helps you move faster, that's a signal you know how to leverage modern tooling, which is exactly what we want.

Good Luck, see you at the finish line!

Engineering Team

Words of wisdom from Asif:

"Don't overthink, over-complicate, or over-engineer."

"How do you eat an elephant? Bite by Bite or Byte by Byte!?"